Code Reference: cloudflare/portfolio-ai-worker.js

A source-level explanation with function details, file communication, variables, endpoints, and debugging notes.

Item	Explanation
File	cloudflare/portfolio-ai-worker.js
Purpose	Cloudflare Worker backend for the public portfolio AI assistant and fallback intelligence path.
Lines	276
Size	10,435 bytes
Talks to	Cloudflare/AI layer, GitHub/release layer
Functions	13
Variables/constants	22
API mentions	1
Signals	45

How this file participates in the system

This generated section reads the actual file and points out line numbers, declarations, endpoints, events, source excerpts, and debugging cues.

API endpoints mentioned or called

Item	Explanation
/api/portfolio-ai	Line 228. Handles AI assistant questions.

Important implementation signals

Item	Explanation
Network request at line 2	Frontend-backend communication or public web/API calls. Evidence: "https://mauriceotieno.com",
Network request at line 3	Frontend-backend communication or public web/API calls. Evidence: "https://www.mauriceotieno.com",
Network request at line 4	Frontend-backend communication or public web/API calls. Evidence: "https://otienomaurice.github.io",
Network request at line 5	Frontend-backend communication or public web/API calls. Evidence: "http://localhost:8080",
Network request at line 6	Frontend-backend communication or public web/API calls. Evidence: "http://localhost:8081",
Network request at line 7	Frontend-backend communication or public web/API calls. Evidence: "http://127.0.0.1:8080",
Network request at line 8	Frontend-backend communication or public web/API calls. Evidence: "http://127.0.0.1:8081"
Security or auth at line 11	Publishing protection, API key safety, HTTP security, or user authorization. Evidence: function json(data, status = 200, headers = {}) {
Security or auth at line 14	Publishing protection, API key safety, HTTP security, or user authorization. Evidence: headers: {
Local storage at line 16	Local persistence, draft state, auth cache, update snooze, or browser-side caching. Evidence: "Cache-Control": "no-store",
Security or auth at line 17	Publishing protection, API key safety, HTTP security, or user authorization. Evidence: ...headers
Security or auth at line 30	Publishing protection, API key safety, HTTP security, or user authorization. Evidence: function corsHeaders(request, env) {
Security or auth at line 31	Publishing protection, API key safety, HTTP security, or user authorization. Evidence: const origin = request.headers.get("Origin") || "";
Security or auth at line 36	Publishing protection, API key safety, HTTP security, or user authorization. Evidence: "Access-Control-Allow-Headers": "Content-Type",
Cloudflare or AI at line 54	Public AI assistant, Worker deployment, or model-provider fallback behavior. Evidence: function extractOpenAiText(data = {}) {

Item	Explanation
Security or auth at line 89	Publishing protection, API key safety, HTTP security, or user authorization. Evidence: "Never invent portfolio projects, credentials, files, test results, or private repository access.",
Cloudflare or AI at line 121	Public AI assistant, Worker deployment, or model-provider fallback behavior. Evidence: async function callOpenAi(body, env) {
Cloudflare or AI at line 122	Public AI assistant, Worker deployment, or model-provider fallback behavior. Evidence: const model = env.OPENAI_MODEL || "gpt-4.1-mini";
Cloudflare or AI at line 124	Public AI assistant, Worker deployment, or model-provider fallback behavior. Evidence: model,
Cloudflare or AI at line 135	Public AI assistant, Worker deployment, or model-provider fallback behavior. Evidence: max_output_tokens: Number(env.OPENAI_MAX_OUTPUT_TOKENS || 1000)
Cloudflare or AI at line 138	Public AI assistant, Worker deployment, or model-provider fallback behavior. Evidence: const response = await fetch("https://api.openai.com/v1/responses", {
Security or auth at line 140	Publishing protection, API key safety, HTTP security, or user authorization. Evidence: headers: {
Cloudflare or AI at line 141	Public AI assistant, Worker deployment, or model-provider fallback behavior. Evidence: "Authorization": `Bearer ${env.OPENAI_API_KEY}`,
Cloudflare or AI at line 151	Public AI assistant, Worker deployment, or model-provider fallback behavior. Evidence: error: data?.error?.message || "OpenAI request failed."
Cloudflare or AI at line 156	Public AI assistant, Worker deployment, or model-provider fallback behavior. Evidence: answer: extractOpenAiText(data),
Cloudflare or AI at line 157	Public AI assistant, Worker deployment, or model-provider fallback behavior. Evidence: model
Cloudflare or AI at line 166	Public AI assistant, Worker deployment, or model-provider fallback behavior. Evidence: error: "Cloudflare Workers AI binding is not available for this Worker."
Cloudflare or AI at line 170	Public AI assistant, Worker deployment, or model-provider fallback behavior. Evidence: const model = env.WORKERS_AI_MODEL || "@cf/meta/llama-3.2-3b-instruct";
Cloudflare or AI at line 171	Public AI assistant, Worker deployment, or model-provider fallback behavior. Evidence: const data = await env.AI.run(model, {
Cloudflare or AI at line 173	Public AI assistant, Worker deployment, or model-provider fallback behavior. Evidence: max_tokens: Number(env.OPENAI_MAX_OUTPUT_TOKENS || 1000)
Cloudflare or AI at line 178	Public AI assistant, Worker deployment, or model-provider fallback behavior. Evidence: model
Cloudflare or AI at line 210	Public AI assistant, Worker deployment, or model-provider fallback behavior. Evidence: "The full AI model is not reachable from this Worker right now, so this is a fallback answer from the available context."
Cloudflare or AI at line 214	Public AI assistant, Worker deployment, or model-provider fallback behavior. Evidence: "I can answer that in a general way, but the full AI model is not reachable from this Worker right now.",
Network request at line 223	Frontend-backend communication or public web/API calls. Evidence: async fetch(request, env) {
Security or auth at line 224	Publishing protection, API key safety, HTTP security, or user authorization. Evidence: const cors = corsHeaders(request, env);
Security or auth at line 225	Publishing protection, API key safety, HTTP security, or user authorization. Evidence: if (request.method === "OPTIONS") return new Response(null, { status: 204, headers: cors });
Cloudflare or AI at line 228	Public AI assistant, Worker deployment, or model-provider fallback behavior. Evidence: if (request.method !== "POST" || !url.pathname.endsWith("/api/portfolio-ai")) {
Cloudflare or AI at line 243	Public AI assistant, Worker deployment, or model-provider fallback behavior. Evidence: let provider = env.OPENAI_API_KEY ? "openai" : "cloudflare-workers-ai";
Cloudflare or AI at line 244	Public AI assistant, Worker deployment, or model-provider fallback behavior. Evidence: let result = env.OPENAI_API_KEY
Cloudflare or AI at line 245	Public AI assistant, Worker deployment, or model-provider fallback behavior. Evidence: ? await callOpenAi({ ...body, question }, env)
Cloudflare or AI at line 247	Public AI assistant, Worker deployment, or model-provider fallback behavior. Evidence: if (!result.ok && env.OPENAI_API_KEY && env.AI) {
Cloudflare or AI at line 248	Public AI assistant, Worker deployment, or model-provider fallback behavior. Evidence: provider = "cloudflare-workers-ai";
Cloudflare or AI at line 254	Public AI assistant, Worker deployment, or model-provider fallback behavior. Evidence: model: "worker-fallback",
Cloudflare or AI at line 262	Public AI assistant, Worker deployment, or model-provider fallback behavior. Evidence: model: result.model,
Cloudflare or AI at line 269	Public AI assistant, Worker deployment, or model-provider fallback behavior. Evidence: model: "worker-fallback",

Important constants and variables

Item	Explanation
defaultAllowedOrigins	Line 1: const defaultAllowedOrigins = [
origin	Line 31: const origin = request.headers.get("Origin") || "";
allowOrigin	Line 32: const allowOrigin = allowedOrigins(env).includes(origin) ? origin : defaultAllowedOrigins[0];
parts	Line 56: const parts = [];
messages	Line 96: const messages = [{ role: "system", content: buildInstructions() }];
model	Line 122: const model = env.OPENAI_MODEL || "gpt-4.1-mini";
payload	Line 123: const payload = {
response	Line 138: const response = await fetch("https://api.openai.com/v1/responses", {
data	Line 146: const data = await response.json().catch(() => ({}));
model	Line 170: const model = env.WORKERS_AI_MODEL || "@cf/meta/llama-3.2-3b-instruct";
data	Line 171: const data = await env.AI.run(model, {
question	Line 183: const question = String(body.question || "").trim();
clean	Line 184: const clean = question.toLowerCase();
context	Line 185: const context = body.context || {};
projects	Line 186: const projects = Array.isArray(context.projects) ? context.projects : [];
owner	Line 187: const owner = context.owner || context.profile?.displayName || "Maurice Otieno";
cors	Line 224: const cors = corsHeaders(request, env);
url	Line 227: const url = new URL(request.url);
body	Line 232: let body = {};
question	Line 239: const question = clamp(body.question, 1200).trim();
provider	Line 243: let provider = env.OPENAI_API_KEY ? "openai" : "cloudflare-workers-ai";
result	Line 244: let result = env.OPENAI_API_KEY

JSON/YAML-style keys detected

Item	Explanation
Content-Type	Line 15: "Content-Type": "application/json; charset=utf-8",
Cache-Control	Line 16: "Cache-Control": "no-store",
Access-Control-Allow-Origin	Line 34: "Access-Control-Allow-Origin": allowOrigin,
Access-Control-Allow-Methods	Line 35: "Access-Control-Allow-Methods": "POST, OPTIONS",
Access-Control-Allow-Headers	Line 36: "Access-Control-Allow-Headers": "Content-Type",
Access-Control-Max-Age	Line 37: "Access-Control-Max-Age": "86400",
Vary	Line 38: "Vary": "Origin"
Authorization	Line 141: "Authorization": `Bearer ${env.OPENAI_API_KEY}`,
Content-Type	Line 142: "Content-Type": "application/json"

Function-by-function detail

json - lines 11-20

Item	Explanation
Source location	Lines 11-20 (10 source lines).
Purpose	Named function in the repository. Inspect the surrounding lines to learn its exact inputs and outputs.
Declaration	function json(data, status = 200, headers = {}) {
Touches	Mostly local calculations or local UI state.
Calls detected	Response, stringify
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	0 await expression(s), 1 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```
11: function json(data, status = 200, headers = {}) {
12:   return new Response(JSON.stringify(data, null, 2), {
13:     status,
14:     headers: {
15:       "Content-Type": "application/json; charset=utf-8",
16:       "Cache-Control": "no-store",
17:       ...headers
18:     }
19:   });
20: }
```

allowedOrigins - lines 22-28

Item	Explanation
Source location	Lines 22-28 (7 source lines).
Purpose	Named function in the repository. Inspect the surrounding lines to learn its exact inputs and outputs.
Declaration	function allowedOrigins(env) {
Touches	Mostly local calculations or local UI state.
Calls detected	split, map, trim, filter, concat
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	0 await expression(s), 1 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```
22: function allowedOrigins(env) {
23:   return String(env.ALLOWED_ORIGINS || "")
24:     .split(",")
25:     .map((origin) => origin.trim())
26:     .filter(Boolean)
27:     .concat(defaultAllowedOrigins);
28: }
```

corsHeaders - lines 30-40

Item	Explanation
Source location	Lines 30-40 (11 source lines).
Purpose	Named function in the repository. Inspect the surrounding lines to learn its exact inputs and outputs.
Declaration	function corsHeaders(request, env) {
Touches	Mostly local calculations or local UI state.
Calls detected	get, allowedOrigins, includes
API endpoints	None detected inside this function body.

Item	Explanation
Storage keys	No local/session storage keys detected.
Async/return clues	0 await expression(s), 1 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```
30: function corsHeaders(request, env) {
31:   const origin = request.headers.get("Origin") || "";
32:   const allowOrigin = allowedOrigins(env).includes(origin) ? origin : defaultAllowedOrigins[0];
33:   return {
34:     "Access-Control-Allow-Origin": allowOrigin,
35:     "Access-Control-Allow-Methods": "POST, OPTIONS",
36:     "Access-Control-Allow-Headers": "Content-Type",
37:     "Access-Control-Max-Age": "86400",
38:     "Vary": "Origin"
39:   };
40: }
```

clamp - lines 42-44

Item	Explanation
Source location	Lines 42-44 (3 source lines).
Purpose	Named function in the repository. Inspect the surrounding lines to learn its exact inputs and outputs.
Declaration	function clamp(value, maxLength) {
Touches	Mostly local calculations or local UI state.
Calls detected	slice
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	0 await expression(s), 1 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```
42: function clamp(value, maxLength) {
43:   return String(value || "").slice(0, maxLength);
44: }
```

cleanConversation - lines 46-52

Item	Explanation
Source location	Lines 46-52 (7 source lines).
Purpose	Named function in the repository. Inspect the surrounding lines to learn its exact inputs and outputs.
Declaration	function cleanConversation(conversation = []) {
Touches	Mostly local calculations or local UI state.
Calls detected	isArray, slice, map, clamp, filter
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	0 await expression(s), 2 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```
46: function cleanConversation(conversation = []) {
47:   if (!Array.isArray(conversation)) return [];
48:   return conversation.slice(-8).map((item) => ({
49:     role: item?.role === "assistant" ? "assistant" : "user",
50:     content: clamp(item?.content || item?.text || "", 1400)
```

```
51:   })).filter((item) => item.content);
52: }
```

extractOpenAiText - lines 54-64

Item	Explanation
Source location	Lines 54-64 (11 source lines).
Purpose	Named function in the repository. Inspect the surrounding lines to learn its exact inputs and outputs.
Declaration	function extractOpenAiText(data = {}) {
Touches	Mostly local calculations or local UI state.
Calls detected	trim, push, join
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	0 await expression(s), 2 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```
54: function extractOpenAiText(data = {}) {
55:   if (typeof data.output_text === "string" && data.output_text.trim()) return data.output_text.trim();
56:   const parts = [];
57:   for (const item of data.output || []) {
58:     for (const content of item.content || []) {
59:       if (typeof content.text === "string") parts.push(content.text);
60:       if (typeof content.output_text === "string") parts.push(content.output_text);
61:     }
62:   }
63:   return parts.join("\n").trim();
64: }
```

extractWorkersAiText - lines 66-79

Item	Explanation
Source location	Lines 66-79 (14 source lines).
Purpose	Named function in the repository. Inspect the surrounding lines to learn its exact inputs and outputs.
Declaration	function extractWorkersAiText(data = {}) {
Touches	Mostly local calculations or local UI state.
Calls detected	trim, isArray, map, filter, join
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	0 await expression(s), 9 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```
66: function extractWorkersAiText(data = {}) {
67:   if (typeof data.response === "string" && data.response.trim()) return data.response.trim();
68:   if (typeof data.result?.response === "string" && data.result.response.trim()) return
data.result.response.trim();
69:   if (typeof data.output_text === "string" && data.output_text.trim()) return data.output_text.trim();
70:   if (Array.isArray(data.output)) {
71:     return data.output.map((item) => {
72:       if (typeof item === "string") return item;
73:       if (typeof item?.text === "string") return item.text;
74:       if (typeof item?.content === "string") return item.content;
75:       return "";
76:     }).filter(Boolean).join("\n").trim();
77:   }
78:   return "";
79: }
```

buildInstructions - lines 81-93

Item	Explanation
Source location	Lines 81-93 (13 source lines).
Purpose	Creates an object, UI structure, process, payload, or generated artifact.
Declaration	function buildInstructions() {
Touches	authentication/security data
Calls detected	join
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	0 await expression(s), 1 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```
   81: function buildInstructions() {
   82:   return [
   83:     "You are the AI assistant for a public engineering portfolio website.",
   84:     "Answer naturally like a helpful technical mentor and recruiter-facing portfolio guide.",
   85:     "If the question is general, answer the concept directly before mentioning portfolio context.",
   86:     "If the question is about the portfolio owner, projects, files, resume, tools, links, or sections,
answer from the supplied portfolio context.",
   87:     "Use recent conversation to keep a conversational flow. Greetings should receive short greetings.",
   88:     "Only include portfolio links when they are directly relevant to the visitor's question.",
   89:     "Never invent portfolio projects, credentials, files, test results, or private repository access.",
   90:     "If a requested detail is not in the context, say what is missing and give a useful next step.",
   91:     "Keep answers concise, readable, and specific. Use bullets when they make the answer easier to scan."
   92:   ].join("\n");
   93: }
```

buildMessages - lines 95-105

Item	Explanation
Source location	Lines 95-105 (11 source lines).
Purpose	Creates an object, UI structure, process, payload, or generated artifact.
Declaration	function buildMessages(body = {}) {
Touches	Mostly local calculations or local UI state.
Calls detected	buildInstructions, cleanConversation, push, buildUserPrompt
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	0 await expression(s), 1 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```
   95: function buildMessages(body = {}) {
   96:   const messages = [{ role: "system", content: buildInstructions() }];
   97:   for (const item of cleanConversation(body.conversation)) {
   98:     messages.push({
   99:       role: item.role === "assistant" ? "assistant" : "user",
  100:       content: item.content
  101:     });
  102:   }
  103:   messages.push({ role: "user", content: buildUserPrompt(body) });
  104:   return messages;
  105: }
```

buildUserPrompt - lines 107-119

Item	Explanation
Source location	Lines 107-119 (13 source lines).
Purpose	Creates an object, UI structure, process, payload, or generated artifact.

Item	Explanation
Declaration	function buildUserPrompt(body = {}) {
Touches	Mostly local calculations or local UI state.
Calls detected	clamp, stringify, cleanConversation, join
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	0 await expression(s), 1 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```
107: function buildUserPrompt(body = {}) {
108:   return [
109:     `Visitor question: ${clamp(body.question, 1200)}`,
110:     `Question intent: ${clamp(body.intent || "portfolio_specific", 80)}`,
111:     `Web/public lookup requested by browser: ${body.allowWebSearch === true ? "yes" : "no"}`,
112:     "",
113:     "Recent conversation JSON:",
114:     clamp(JSON.stringify(cleanConversation(body.conversation), null, 2), 6000),
115:     "",
116:     "Portfolio context JSON:",
117:     clamp(JSON.stringify(body.context || {}, null, 2), 20000)
118:   ].join("\n");
119: }
```

callOpenAi - lines 121-145

Item	Explanation
Source location	Lines 121-145 (25 source lines).
Purpose	Named function in the repository. Inspect the surrounding lines to learn its exact inputs and outputs.
Declaration	async function callOpenAi(body, env) {
Touches	network/API requests, authentication/security data
Calls detected	buildInstructions, buildUserPrompt, fetch, stringify
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	1 await expression(s), 0 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```
121: async function callOpenAi(body, env) {
122:   const model = env.OPENAI_MODEL || "gpt-4.1-mini";
123:   const payload = {
124:     model,
125:     input: [
126:       {
127:         role: "developer",
128:         content: [{ type: "input_text", text: buildInstructions() }]
129:       },
130:       {
131:         role: "user",
132:         content: [{ type: "input_text", text: buildUserPrompt(body) }]
133:       }
134:     ],
135:     max_output_tokens: Number(env.OPENAI_MAX_OUTPUT_TOKENS || 1000)
136:   };
137:
138:   const response = await fetch("https://api.openai.com/v1/responses", {
139:     method: "POST",
140:     headers: {
141:       "Authorization": `Bearer ${env.OPENAI_API_KEY}`,
142:       "Content-Type": "application/json"
143:     },
144:     body: JSON.stringify(payload)
145:   });
```

callWorkersAi - lines 161-180

Item	Explanation
Source location	Lines 161-180 (20 source lines).
Purpose	Named function in the repository. Inspect the surrounding lines to learn its exact inputs and outputs.
Declaration	async function callWorkersAi(body, env) {
Touches	authentication/security data
Calls detected	run, buildMessages, extractWorkersAiText
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	1 await expression(s), 2 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```
161: async function callWorkersAi(body, env) {
162:   if (!env.AI || typeof env.AI.run !== "function") {
163:     return {
164:       ok: false,
165:       status: 503,
166:       error: "Cloudflare Workers AI binding is not available for this Worker."
167:     };
168:   }
169:
170:   const model = env.WORKERS_AI_MODEL || "@cf/meta/llama-3.2-3b-instruct";
171:   const data = await env.AI.run(model, {
172:     messages: buildMessages(body),
173:     max_tokens: Number(env.OPENAI_MAX_OUTPUT_TOKENS || 1000)
174:   });
175:   return {
176:     ok: true,
177:     answer: extractWorkersAiText(data),
178:     model
179:   };
180: }
```

fallbackWorkerAnswer - lines 182-220

Item	Explanation
Source location	Lines 182-220 (39 source lines).
Purpose	Named function in the repository. Inspect the surrounding lines to learn its exact inputs and outputs.
Declaration	function fallbackWorkerAnswer(body = {}) {
Touches	filesystem
Calls detected	trim, toLowerCase, isArray, slice, map, filter, join, b
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	0 await expression(s), 4 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```
182: function fallbackWorkerAnswer(body = {}) {
183:   const question = String(body.question || "").trim();
184:   const clean = question.toLowerCase();
185:   const context = body.context || {};
186:   const projects = Array.isArray(context.projects) ? context.projects : [];
187:   const owner = context.owner || context.profile?.displayName || "Maurice Otieno";
188:
189:   if (/^(hi|hello|hey|yo)\b/.test(clean)) {
190:     return `Hi, what can I do for you? I can help with ${owner}'s projects, files, resume links, or
related electronics concepts.`;
191:   }
192:   if (/\bembedded systems?\b/.test(clean)) {
193:     return [
```

```
      "Embedded systems is a field of engineering where software is built into dedicated hardware to control a device, measure signals, communicate with peripherals, or respond to real-time events.",
      "",
      "Typical embedded work combines microcontrollers or processors, firmware, timing, interrupts, communication buses, sensors, actuators, and hardware validation.",
      "",
      projects.length
        ? `In this portfolio, I would connect that explanation back to saved projects such as ${projects.slice(0, 3).map((project) => project.title || project.name).filter(Boolean).join(", ")} when those projects are relevant.`
        : "When project context is available, I connect the explanation to the saved project evidence."
    ].join("\n");
  }
  if (projects.length && /\b(project|portfolio|maurice|file|github|resume|tool|design|test)\b/.test(clean)) {
    return [
      `I can use the portfolio context for ${owner}.`,
      "",
      "Relevant saved project areas include:",
      ...projects.slice(0, 6).map((project) => `- ${project.title || project.name || "Untitled project"}`),
      "",
      "The full AI model is not reachable from this Worker right now, so this is a fallback answer from the available context."
    ].join("\n");
  }
  return [
    "I can answer that in a general way, but the full AI model is not reachable from this Worker right now.",
    "",
    "A useful answer should define the concept, explain the main parts, give a practical example, and connect it to a project artifact when portfolio context is relevant.",
    "",
    `Question received: ${question}`
  ].join("\n");
}
```

Representative opening snippet

```
const defaultAllowedOrigins = [
  "https://mauriceotieno.com",
  "https://www.mauriceotieno.com",
  "https://otienomaurice.github.io",
  "http://localhost:8080",
  "http://localhost:8081",
  "http://127.0.0.1:8080",
  "http://127.0.0.1:8081"
];

function json(data, status = 200, headers = {}) {
  return new Response(JSON.stringify(data, null, 2), {
    status,
    headers: {
      "Content-Type": "application/json; charset=utf-8",
      "Cache-Control": "no-store",
      ...headers
    }
  });
}

function allowedOrigins(env) {
  return String(env.ALLOWED_ORIGINS || "")
    .split(",")
    .map((origin) => origin.trim())
    .filter(Boolean)
    .concat(defaultAllowedOrigins);
}

function corsHeaders(request, env) {
  const origin = request.headers.get("Origin") || "";
  const allowOrigin = allowedOrigins(env).includes(origin) ? origin : defaultAllowedOrigins[0];
  return {
    "Access-Control-Allow-Origin": allowOrigin,
    "Access-Control-Allow-Methods": "POST, OPTIONS",
    "Access-Control-Allow-Headers": "Content-Type",
    "Access-Control-Max-Age": "86400",
    "Vary": "Origin"
  };
}

function clamp(value, maxLength) {
  return String(value || "").slice(0, maxLength);
}

function cleanConversation(conversation = []) {
  if (!Array.isArray(conversation)) return [];
  return conversation.slice(-8).map((item) => ({
    role: item?.role === "assistant" ? "assistant" : "user",
    content: clamp(item?.content || item?.text || "", 1400)
  })).filter((item) => item.content);
```

```
}

function extractOpenAiText(data = {}) {
  if (typeof data.output_text === "string" && data.output_text.trim()) return data.output_text.trim();
  const parts = [];
  for (const item of data.output || []) {
    for (const content of item.content || []) {
      if (typeof content.text === "string") parts.push(content.text);
      if (typeof content.output_text === "string") parts.push(content.output_text);
    }
  }
  return parts.join("\n").trim();
}

function extractWorkersAiText(data = {}) {
  if (typeof data.response === "string" && data.response.trim()) return data.response.trim();
  if (typeof data.result?.response === "string" && data.result.response.trim()) return
data.result.response.trim();
  if (typeof data.output_text === "string" && data.output_text.trim()) return data.output_text.trim();
  if (Array.isArray(data.output)) {
    return data.output.map((item) => {
      if (typeof item === "string") return item;
```